

Problem and Motivation

Monte Carlo simulation in financial applications, and in particular, to the Heston volatility model, involves drawing from a process in which the underlying asset and variance dynamics are defined in continuous time. With most financial instruments (such as European call and put options) relying on the terminal value of the asset, pricing these derivatives requires the full evolutive context of the asset's trajectory up to the terminal horizon. In practice however, exact simulation schemes rarely exist in close-form and hence require approximations on a discrete time-grid. This introduces two distinct sources of error: sampling error, from using a finite number of simulated paths; and discretization bias, from replacing the true stochastic differential equation (SDE) with a time-stepping approximation. The first decreases with additional simulation effort, while the second persists unless the time grid is refined. As a result, standard discretization schemes require trade-off between the number of time-steps and the number of sample paths, and the optimal balance is generally unknown since the rate at which discretization bias vanishes depends entirely on the scheme and model dynamics.

Exact Versus Discrete Simulation

Discretized methods such as Euler-type schemes are straightforward to implement and often computationally inexpensive per path, but they produce biased estimators unless the time step is taken to be sufficiently small. Reducing this bias requires a finer grid, which increases computational cost and can slow the overall rate of convergence under a fixed computational budget. Exact simulation methods avoid this problem by sampling from the true transition law of the model rather than using an approximation. Their main advantage is the elimination of discretization bias, so that only the Monte Carlo error remains. The trade-off in this regard, is that exact methods are typically mathematically demanding, since they require nontrivial distributional results and more sophisticated sampling procedures. With an exact method available, one would expect bias and error measures to be less than those produced by a discrete method. In any case however, these error measures are expected to decrease as the number of Monte Carlo simulations increase, and in the discrete case, as the discrete time intervals are made smaller.

Broadie & Kaya on the Heston Model

Seminal results presented by Broadie & Kaya (2006), illustrate an exact simulation method for the well-known Heston Stochastic Volatility model by exploring its affine structure. The key idea is to avoid discretizing the coupled stock-price and variance system and instead to decompose the simulation problem into quantities whose conditional laws are tractable. This is done under the objective of producing:

- (1) Unbiased estimators of European call option prices under stochastic volatility; and
- (2) Producing these estimators under a given computational budget s .

It is shown moreover in the paper, that for time-budget s ; their proposed exact method achieves a convergence rate that is deterministically bounded on the order $O\left(s^{-\frac{1}{2}}\right)$, which is faster than the Euler discretization error of $O\left(s^{-\frac{1}{3}}\right)$ (or slower). The consequence of which, being that the exact method implies a more efficient reduction in error, under constrained computational cost.

First, they discuss how the terminal variance can be sampled exactly, because the square-root variance process has a known transition distribution. Specifically, a scaled noncentral chi-square law. In particular, the variance at time t given the variance at time u is given by:

$$V_t = \frac{\sigma_v^2(1 - e^{-\kappa(t-u)})}{4\kappa} \chi_a^2\left(\frac{4\kappa e^{-\kappa(t-u)}}{\sigma_v^2(1 - e^{-\kappa(t-u)})} \times V_u\right) \quad (1)$$

with degrees of freedom:

$$d = \frac{4\theta\kappa}{\sigma_v^2} \quad (2)$$

Second, the time integrated variance, although not available in close-form density, has a known conditional characteristic function which allows it to be sampled using Fourier inversion. Third, conditional on the terminal variance and integrated variance, the remaining stochastic integral terms can be recovered, and the stock price can then be sampled from its exact conditional lognormal distribution. Thus, the Broadie–Kaya algorithm produces samples from the true Heston model without time discretization, yielding unbiased Monte Carlo estimators and improved error behavior relative to standard discretization-based simulation.

Although these results are tractable, a central difficulty in the Broadie-Kaya algorithm arises from the treatment of the time integrated variance:

$$I = \int_u^t V_s ds \quad (3)$$

which is the path dependent functional of the variance process and is inherently random. Unlike the terminal variance, whose transition distribution is known in close form, the integrated variance does not admit a tractable density and depends on the entire trajectory of the stochastic volatility between two time points. This makes it nontrivial to simulate directly. Although its conditional characteristic function $\Phi_I(a)$ is available due to the affine-structure of the Cox-Ingersoll-Ross process, recovering samples from this distribution requires numerical inversion techniques (in this case, Fourier inversion). Specifically, the main object involves numerical inversion of $\Phi_I(a)$ to obtain the distribution function:

$$\Pr(I \leq x) = \frac{1}{\pi} \int_{-\infty}^{\infty} \frac{\sin(ux)}{u} \Phi_I(u) du \quad (4)$$

which is approximated by the trapezoidal rule as:

$$\Pr(I \leq x) \approx \frac{hx}{\pi} + \frac{2}{\pi} \sum_{j=1}^{\infty} \frac{\sin(hjx)}{j} \text{Re}[\Phi(hj)] - e_d(h) \quad (5)$$

This step introduces significant computational complexity and is widely regarded as the computational ‘bottleneck’ of Broadie & Kaya’s (2006) proposed framework. Truncation of (5) is possible, and a proposed stopping mechanism is discussed in the paper. Simulation from (5) is achieved using the inverse transform method, using a second-order newton iteration scheme.

The Broadie & Kaya Algorithm

Having established that exact simulation in the Heston framework can be achieved by decomposing the joint dynamics into tractable conditional components, the Broadie–Kaya methodology proceeds by sequentially sampling from these components in a manner that preserves the true underlying distribution. The resulting procedure is a structured, multi-step algorithm which produces exact samples of the terminal asset price under stochastic volatility, and hence unbiased Monte Carlo estimators of derivative prices.

Let V_u denote the variance at time u , and simulate forward to time T . The algorithm proceeds as follows:

- (1) Draw $V_T|V_u$ exactly from its known transition distribution
- (2) Draw $I = \int_u^T V_s ds$ conditional on V_u and V_T using numerical inversion of its conditional characteristic function via Fourier methods.
- (3) Conditional on V_u, V_T , and I ; reconstruct the stochastic integral term associated with the variance process.
- (4) Conditional on the sampled quantities above, draw the terminal log-stock price from its exact conditional distribution, which is gaussian.
- (5) Exponentiation yields the terminal asset price.

Replication Results

Tables 1 and 2 in Broadie and Kaya (2006) report numerical simulation results for pricing a European call option under the stochastic volatility model, comparing the exact simulation method against Euler discretization across increasing numbers of simulation trials, with attention to RMS error, bias, standard error, and computing time. Table 1 considers the baseline Heston parameterization drawn from Duffie et al. (2000), while Table 2 examines a parameter setting in which Euler discretization performs especially poorly because discretization bias remains substantial even with many time steps. Table 3 then isolates the effect of the Euler time-step choice by reporting bias values under alternative time-step allocations— this illustrates that discretization bias can dominate statistical error and converge more slowly than the standard first-order intuition would suggest. Accordingly, the results below have been generated to replicate the numerical results reported in Tables 1–3 of Broadie and Kaya (2006), with emphasis on reproducing the behavior of the exact simulation and Euler discretization methods in terms of bias, RMS error, and computational efficiency.

In this these results, C++ was used to implement the algorithm and Monte-Carlo replication, because the computational burden of the algorithm is concentrated in nested operations across many Monte-Carlo paths. In particular, the code repeatedly evaluates the conditional characteristic function through `evaluate_integrated_variance_cf()`, manages branch-sensitive Bessel-function terms through `evaluate_branch_corrected_bessel()`, constructs the truncated Fourier grid using `compute_truncated_cf_grid()`, solves the nonlinear inverse-CDF problem in `sample_integrated_variance()`, and then propagates the sampled quantities through `sample_terminal_stock_price_exact()` and `estimate_call_price_exact()` to obtain option-price estimates and error measures. In Python, such operations are comparatively slow and may exceed practical run-time limits. Moreover, Broadie and Kaya themselves implement their numerical experiments in C++, so the present implementation is consistent with the computational strategy used in the original paper to make exact simulation feasible under a constrained computational budget.

As before, the main issue occurs when sampling the integrated variance (equation 3), conditional on the initial and terminal variance values. The paper establishes this step theoretically through Fourier inversion of the conditional characteristic function; while this implementation applies it by numerically recovering the CDF and then applying inverse transform sampling. For each path, a uniform random variable $U \sim \text{Unif}(0,1)$ is generated, and the algorithm solves:

$$F_I(x) - U = 0 \quad (6)$$

where the CDF $F_I(x)$ is obtained from a truncated Fourier inversion formula. To make this root-finding problem computationally stable, the search is initialized using a moment-matched normal approximation: the conditional mean and variance of the integrated variance are used to construct a Gaussian proxy distribution, and its quantile provides an initial guess for x . The code then attempts Newton–Raphson iteration, using a branch-corrected Bessel-function evaluation to keep the characteristic-function calculation on the appropriate complex branch. If Newton updates fail to converge, leave the admissible region, or become unstable because of the oscillatory Fourier terms, the routine falls back to a bracketing/bisection method. Despite that this differs from the Broadie Kaya paper, the target distribution remains the Broadie–Kaya conditional law. The hybrid Newton–bisection strategy makes the inversion robust under finite-precision arithmetic, high volatility, long maturities, and branch-sensitive complex-function evaluations. With more extreme inputs—large vol-of-vol, long maturities, near-zero variance states, high correlation sensitivity, or tail uniforms near 0 or 1—the code would be expected to spend more time in fallback routines and branch corrections, so runtime and numerical instability may increase even though the estimator remains conceptually exact.

Tables 1 and 2 represent the exact and Euler results, respectively, corresponding to Table 1 in Broadie and Kaya (2006); while Tables 3 and 4 represent the same comparison for Broadie and Kaya’s Table 2. Across Figures 1–4, the replicated results reproduce the central pattern of the original paper. For the first parameter setting, the exact method exhibits a smooth and systematic reduction in RMSE as the number of trials increases (0.0744 at 10000 paths, to 0.00232 at 10240000 paths). The Euler method also improves in RMSE with additional trials and time steps, but its error decreases less smoothly because the estimator remains affected by discretization bias (0.221 at 10000 paths to 0.0190 at 10240000 paths). For the second parameter setting, the contrast becomes stronger: the exact method again shows a smooth decrease in RMSE (0.548 at 10000 paths, to 0.0180 at 10240000 paths), whereas the Euler method retains substantial bias even at the largest simulation sizes (2.479 at 10000 paths, to 0.566 at 10240000 paths). Overall, the figures support Broadie and Kaya’s main conclusion that exact simulation achieves the expected $O\left(M^{-\frac{1}{2}}\right)$ convergence by eliminating discretization bias, while Euler discretization converges more slowly because both sampling error and discretization bias contribute to the RMSE. The shorter computing times in Figures 1–4 are expected given that Broadie and Kaya’s original results were published in 2006, before substantial improvements to computer hardware causing decreases in processing times. Nevertheless, the relative trade-off remains the same: Euler is faster for small simulations, but exact simulation becomes more attractive as higher numerical accuracy is required.

Relative to Table 3 in Broadie and Kaya (2006), table 5 shows the same fundamental pattern: the Euler estimator is most impacted by discretization bias arising from the time-discretized approximation of the continuous-time Heston dynamics. As the number of simulation trials increases, the standard error declines systematically, yet the bias decreases much more slowly and remains sensitive to the chosen time-step. Under the parameters chosen for their first parametrization (Table 1), increasing the grid from $0.1\sqrt{N}$ to $\sqrt{N}$, and then to $10\sqrt{N}$, produces a clear reduction in bias, but the bias is still non-negligible relative to the standard error except at the finest grids. Under the Table 2 parameterization, this issue is substantially more pronounced: even when the number of time steps is set to $10\sqrt{N}$, the bias remains large, indicating that the Euler approximation struggles in parameter regimes where the variance process is more difficult to simulate accurately. The replication of these results confirms Broadie and Kaya’s conclusion that Euler discretization involves a bias–variance–cost trade-off: increasing N reduces statistical uncertainty, but unless the time grid is refined sufficiently, the estimator may converge toward a biased quantity rather than the true option value.

Tables:

Table 1: Exact Results (Table 1)

Exact_table1

Number of Trials	RMS Error (SE)	Time (Sec)
10000	0.0743974244	0.8038318330
40000	0.0369170673	3.1417168750
160000	0.0185771948	12.6717517500
640000	0.0092578821	52.1698184170
2560000	0.0046314327	212.3517667090
10240000	0.0023162780	847.3951691670

Table 2: Euler Results (Table 1)

Euler_table1

M	Time_Steps	Price	Bias	Standard Error	RMSE	Time (Sec)
10000	100	7.0129587313	0.2068587313	0.0778962726	0.2210392815	0.0330957920
40000	200	6.8834326429	0.0773326429	0.0379916845	0.0861609294	0.1248111250
160000	400	6.8884253883	0.0823253883	0.0188845201	0.0844635700	0.9132149580
640000	800	6.8404933451	0.0343933451	0.0093876059	0.0356514983	7.4000515000
2560000	1600	6.8389053462	0.0328053462	0.0046758485	0.0331369023	61.7122532090
10240000	3200	6.8249898782	0.0188898782	0.0023307295	0.0190331237	493.9595210000

Note: In tables 1 and 2, the option parameters were taken as:

$S = 100, K = 100, V_0 = 0.010201, \kappa = 6.21, \theta = 0.019, \sigma_v = 0.61, \rho = -0.7, r = 0.0319, T = 1,$
 $C_{TRUE} = 6.8061$

Table 3: Exact results (Table 2)

Exact_table2

Number of Trials	RMS Error (SE)	Time (Sec)
10000	0.5479739162	3.4416019170
40000	0.2791283094	13.8455054590
160000	0.1396023181	56.3317332090
640000	0.0697188739	227.6459241250
2560000	0.0347433142	909.1288291670
10240000	0.0178986265	3640.8323532500

Table 4: Euler results (Table 2)

Euler_table2

M	Time_Steps	Price	Bias	Standard Error	RMSE	Time (Sec)
10000	100	37.3936963368	2.3938963368	0.6456246918	2.4794295541	0.0257894580
40000	200	36.2061413251	1.2063413251	0.3125461615	1.2461719367	0.1223446670
160000	400	36.2609667331	1.2611667331	0.1540407485	1.2705392875	0.9363915000
640000	800	35.8835818212	0.8837818212	0.0767739294	0.8871102207	7.5711815000
2560000	1600	35.7380246856	0.7382246856	0.0377123494	0.7391873293	61.0926081250
10240000	3200	35.5650892958	0.5652892958	0.0187205483	0.5655991928	488.5753386250

Note: In tables 3 and 4, the option parameters were taken as:

$S = 100$, $K = 100$, $V_0 = 0.09$, $\kappa = 2.00$, $\theta = 0.09$, $\sigma_v = 1$, $\rho = 0.3$, $r = 0.05$, $T = 5$, $C_{TRUE} = 34.9998$

Table 5: Euler Results for Different Time Steps (Table 3)

Table3

Scheme	Number of Trials	Standard Error	Time Steps $0.1\sqrt{tN}$	Bias $0.1\sqrt{tN}$	Time Steps $\sqrt{tN}$	Bias $\sqrt{tN}$	Time Steps $10\sqrt{tN}$	Bias $10\sqrt{tN}$	Time (Sec)
Table 1 Parameters	10000	0.0781477862	10	0.4273226345	100	0.1341890331	1000	0.1826722199	0.1862418330
Table 1 Parameters	40000	0.0382660804	20	0.3345973720	200	0.1392153915	2000	0.0345939696	1.2624402500
Table 1 Parameters	160000	0.0188740416	40	0.2855052286	400	0.0710305473	4000	0.0256373510	10.2682719580
Table 1 Parameters	640000	0.0093781022	80	0.1724750483	800	0.0322172562	8000	0.0183366847	84.1772400000
Table 1 Parameters	2560000	0.0046715729	160	0.1108520426	1600	0.0180521515	16000	0.0112763258	672.1344162080
Table 1 Parameters	10240000	0.0023312040	320	0.0771182192	3200	0.0208883902	32000	0.0046547572	5375.2023257910
Table 2 Parameters	10000	0.6409339465	10	6.3939097384	100	2.5915958763	1000	1.0563874467	0.1638156250
Table 2 Parameters	40000	0.3071610267	20	4.5044446133	200	1.1908977230	2000	0.2968883251	1.3101841670
Table 2 Parameters	160000	0.1551249315	40	3.3032037687	400	1.0359660391	4000	0.3423255957	10.4797334170
Table 2 Parameters	640000	0.0772676858	80	2.4458262184	800	0.9160453313	8000	0.3719202935	83.8473041250
Table 2 Parameters	2560000	0.0377064821	160	1.8521178208	1600	0.6201728927	16000	0.2983058135	670.7229889170
Table 2 Parameters	10240000	0.0186906947	320	1.3513609493	3200	0.5573419294	32000	0.2533033924	5377.1804829580